

Chapter 03

網站前端與後端

Horazon

互動媒體設計

網頁開發的三大支柱

如果要創造一個完整的網頁，我們需要三個核心技術：

1. **HTML** (HyperText Markup Language)
2. **CSS** (Cascading Style Sheets)
3. **JavaScript** (JS)

它們各司其職，缺一不可。

比喻：人體構造

如果把網頁比喻成一個「人」：

- **HTML 是 骨架 (Skeleton)**
 - 決定了人的高矮胖瘦、有沒有頭、有幾隻手。
 - 沒有骨架，人就只是一坨肉。
- **CSS 是 皮膚與衣服 (Skin & Clothing)**
 - 決定了膚色、穿什麼衣服、髮型好不好看。
 - 沒有 CSS，人會長得很醜 (只有白色背景和黑色文字)。
- **JavaScript 是 肌肉與大腦 (Muscle & Brain)**
 - 讓人可以動起來、會說話、會思考。
 - 沒有 JS，人就是個不會動的植物人。

1. HTML (結構層)

HyperText Markup Language (超文本標記語言)

- **不是**程式語言 (Programming Language)，是**標記語言** (Markup Language)。
- 負責告訴瀏覽器：
 - 這是一段標題 (`<h1>`)
 - 這是一張圖片 (`<img>`)
 - 這是一個按鈕 (`<button>`)

```
<!-- 只有結構，沒有樣式 -->
<h1>歡迎光臨</h1>
<p>這是一個樸素的網頁。</p>
<button>點我</button>
```

2. CSS (表現層)

Cascading Style Sheets (階層式樣式表)

- 負責美化 HTML 定義的元素。
- 控制顏色、字體、大小、版面配置 (Layout)、動畫效果。

```
/* 讓標題變紅色，置中 */  
h1 {  
    color: red;  
    text-align: center;  
}  
  
/* 讓按鈕變圓角，背景藍色 */  
button {  
    background-color: blue;  
    color: white;  
    border-radius: 10px;  
}
```

3. JavaScript (行為層)

JavaScript (JS)

- 真正的**程式語言** (有變數、迴圈、邏輯判斷)。
- 負責處理使用者互動、資料運算、與伺服器溝通。

```
// 當按鈕被點擊時...
button.addEventListener('click', function() {
    // 彈出一個視窗
    alert('你點到我了!');
    // 或是改變網頁背景顏色
    document.body.style.backgroundColor = 'yellow';
});
```

MVC 概念介紹

在軟體工程中，為了讓程式碼好維護，我們常使用 **MVC 架構** 來開發。

MVC 是 **Model - View - Controller** 的縮寫。

這種架構將程式分成三個部分，各司其職，互不干擾。

MVC 示意圖

角色	名稱	職責	網頁對應
M	Model (模型)	負責資料的儲存與邏輯 (Data)	HTML (結構/資料) 或是後端資料庫
V	View (視圖)	負責顯示畫面給使用者看 (Display)	CSS (樣式/外觀)
C	Controller (控制器)	負責處理使用者的輸入與互動 (Interaction)	JavaScript (行為/邏輯)

註：在純網頁前端領域，常將 HTML 視為 Model (結構)，CSS 視為 View (表現)，JS 視為 Controller (行為)。但在更複雜的 Web App 中，Model 通常指從伺服器抓回來的 JSON 資料。

MVC 比喻：高級餐廳

- **Model (廚房/食材)**
 - 負責準備食物 (資料)。廚師不管客人長怎樣，只管把菜做好。
- **View (擺盤/裝潢)**
 - 負責把菜端上桌，擺得漂漂亮亮。盤子不管菜好不好吃，只管好看。
- **Controller (服務生)**
 - 負責點餐 (接收使用者輸入)，通知廚房做菜 (呼叫 Model)，再把菜端給客人 (更新 View)。

為什麼要分這麼細？(關注點分離)

想像一下，如果把 HTML, CSS, JS 全部寫在一起...

- **HTML 裡寫樣式：** `<h1 style="color:red; font-size:20px;">`
 - 如果要改所有標題顏色，要改 100 個地方！
- **HTML 裡寫程式：** `<button onclick="alert('Hello')">`
 - 程式碼散落在各處，除錯困難。

分離的好處：

1. **好維護：**改顏色找 CSS，改內容找 HTML，改功能找 JS。
2. **分工合作：**設計師寫 CSS，工程師寫 JS，互不打架。
3. **可重複使用：**一套 CSS 可以套用到 100 個頁面。

前端 vs 後端 (Frontend vs Backend)

網頁開發通常分為兩大領域：

1. 前端 (Frontend) - Client Side

- 使用者看得到的部分。
- 執行在使用者的**瀏覽器** (Browser) 上。
- 技術：HTML, CSS, JavaScript, React, Vue。
- 重點：介面設計、使用者體驗 (UX)、動畫流暢度。

2. 後端 (Backend) - Server Side

- 使用者看不到的部分。
- 執行在遠端的**伺服器** (Server) 上。
- 技術：Python, PHP, Node.js, Java, SQL (資料庫)。
- 重點：資料存取、商業邏輯、資安、伺服器效能。

網頁運作流程 (Request / Response)

當你在瀏覽器輸入網址並按下 Enter 時...

1. **Request (請求)**：瀏覽器 (前端) 像服務生一樣，把你的點單傳給伺服器 (後端)。
2. **Processing (處理)**：伺服器 (後端) 去資料庫找資料 (如：找商品列表)。
3. **Response (回應)**：伺服器把資料 (HTML/JSON) 傳回給瀏覽器。
4. **Rendering (渲染)**：瀏覽器把收到的程式碼變成你看得懂的畫面。

全端開發 (Full Stack)

什麼是全端工程師？

就是 **前端** + **後端** 都會寫的人！

- 他可以一個人完成整個網站的開發。
- 從資料庫設計、伺服器架設，到網頁切版、動畫特效都能搞定。
- **優點**：溝通成本低，開發原型 (Prototype) 快。
- **缺點**：要學的東西非常多，技術廣度大但可能深度不足。

本課程 (WebHalf) 專注於「**前端**」技術 (HTML/CSS/JS)。

3D 模型與網頁的對應 (進階比喻)

回到我們熟悉的遊戲開發領域...

如果要創造一個虛擬角色 (3D Model)：

1. **Mesh (網格模型) = HTML (結構)**
 - 決定角色的形狀。
2. **Texture/Shader (貼圖/材質) = CSS (外觀)**
 - 決定角色看起來的質感與顏色。
3. **Animator/Script (動畫控制器/腳本) = JavaScript (行為)**
 - 決定角色如何移動、攻擊。

Canvas 與 WebGL

在網頁上也能做遊戲！

- **HTML5 Canvas：**
 - 一個神奇的標籤，可以在上面用 JS 畫圖。
 - 可以做 2D 遊戲 (如 Phaser.js)。
- **WebGL：**
 - 網頁版的 OpenGL。
 - 可以運用顯示卡 (GPU) 跑 3D 圖形。
 - **Unity WebGL** 就是用這個技術把遊戲發布到網頁上！

現代網頁開發趨勢

現在的網頁已經不只是「文件」，而是 Web App (網頁應用程式)。

- PWA (Progressive Web App)：讓網頁像 App 一樣可以安裝在手機桌面，還能離線使用。
- SPA (Single Page Application)：像 Gmail 一樣，切換頁面不用重新整理，速度極快。
- No-Code / AI：用 AI 寫程式，或是用 Webflow 等工具拖拉生成網頁。

这也正是我們這門課要學習的核心技能！

總結 (Summary)

1. **三大支柱：**
 - HTML (骨架/結構)
 - CSS (皮膚/樣式)
 - JS (肌肉/行為)
2. **MVC 架構：**讓程式碼職責分離，易於維護。
3. **前後端分離：**
 - 前端負責畫面與互動 (瀏覽器)。
 - 後端負責資料與邏輯 (伺服器)。
4. **全端：**通吃前後端。

網頁標準 (Web Standards)

誰決定 HTML/CSS 怎麼寫？

不是 Google，也不是 Microsoft，而是一個非營利組織：

W3C (World Wide Web Consortium)

- 由全球資訊網發明人 **Tim Berners-Lee** 領導。
- 制定網頁的標準規範 (HTML5, CSS3)。
- **目的**：確保網頁在任何瀏覽器、任何裝置上都能正常顯示。

MDN Web Docs：Mozilla 維護的開發者文件，是我們查語法最權威的地方 (比 W3C 好讀)。

瀏覽器大戰 (Browser Wars)

瀏覽器就像用來讀取網頁的「播放器」，市面上有好幾種：

1. **Google Chrome** (市佔率最高)

- 核心：Blink 引擎。速度快，擴充功能多。

2. **Safari** (Apple 裝置專用)

- 核心：WebKit 引擎。省電，但在開發上有時會有「獨特」的問題。

3. **Microsoft Edge**

- 核心：Blink (現在跟 Chrome 是親兄弟)。內建 Copilot AI。

4. **Firefox**

- 核心：Gecko。強調隱私與自由軟體精神。

響應式網頁設計 (RWD)

Responsive Web Design

- **以前**：電腦版網頁、手機版網頁是分開的 (m.facebook.com)。
- **現在**：這門課要教你的技術！
 - **同一個網址、同一份程式碼**。
 - 網頁會根據螢幕寬度 (電腦/平板/手機) **自動調整排版**。
 - **CSS Media Queries** 是關鍵技術。

Mobile First (行動優先)：現在 70% 的流量來自手機，設計時要先考慮手機體驗！

網址與網域 (URL & Domain)

網頁做好了，要怎麼讓人找到？

1. IP 位址 (如 `140.128.xxx.xxx`)

- 電腦看的門牌號碼，人類記不住。

2. 網域名稱 (Domain Name)

- `google.com` , `hk.edu.tw`
- 人類好記的地址。

3. DNS (Domain Name System)

- 網路上的電話簿，負責把「網域」翻譯成「IP」。

`.com` (商業) / `.edu` (教育) / `.org` (組織) / `.tw` (台灣)

開發者工具 (DevTools)

這是前端工程師最強大的武器！

- 按 F12 或 右鍵 -> 檢查 (Inspect) 開啟。
- Elements：查看網頁的 HTML/CSS 結構 (你可以偷改別人的網頁自嗨，但重新整理就沒了)。
- Console：查看 JavaScript 的錯誤訊息或執行程式碼。
- Network：查看網頁載入了哪些圖片、花了多少時間。

下週我們就會用到它！

下週預告 (Next Week)

工欲善其事，必先利其器。

下週我們將進入**實作環節**：

1. 安裝 VS Code **編輯器** (地表最強寫 code 軟體)。
2. 安裝好用的 Extensions (**擴充套件**)。
3. 寫出人生第一個 `index.html` ！

請確認你的電腦 (或是學校電腦) 可以連上網路並安裝軟體。